

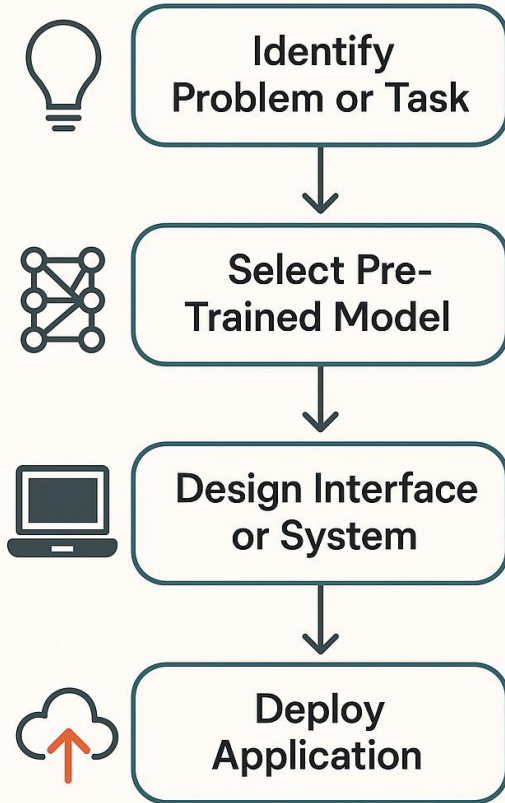
Module 6

AI Engineering

- AI Engineering = building and deploying AI applications/systems in real-world contexts, often utilizing pre-trained models.
- A person who doesn't know gradient descent or backpropagation can still **build powerful AI applications.**
- More focus on engineering/practical integration.
- Many organizations now hire people for AI engineering (not just model research) to bring AI into production

Role	Core Work	Needs Math?
ML Researcher	Design architectures, loss functions, backprop	✓ Yes
ML Engineer	Train + optimize models	⚙️ Some
AI Engineer	Build AI-powered systems using existing models	🚫 Not really
Data Engineer	Move + preprocess data	Minimal
MLOps Engineer	Deploy + monitor models	Minimal

Building AI Applications Without ML Knowledge



**A non-ML person (Software engineer)
can build an AI system step-by-step**

Pre-Trained Foundation Models Changed Everything

Now, giant tech labs (OpenAI, Google, Meta, etc.) have already trained **massive models** — GPT, BERT, CLIP, Whisper, SAM, etc.

You just **use them** via APIs or libraries like Hugging Face.

Then (2015)

Now (2025)

Train CNNs from scratch

Use pretrained CLIP or YOLO

Write loss functions

Call `model.predict()`

Configure GPUs manually

Use cloud inference or APIs

Need calculus, stats, and optimization

Need logic, creativity, and integration skills

Problem statement

Design a **Face-Based Security or Attendance System** that can automatically identify or verify a person's face for granting access or marking attendance — *even if the person is newly enrolled with only one photo (one-shot learning)*.

Example use cases:

- Entry to a secure lab or office.
- Automated classroom attendance.
- Smart home door unlock using face authentication.

Three functionality:

- Face verification
- Face enrolment
- Multiple face recognition (in a single frame)



**Face-recognition-
based home
security access
system**

Robust Face Recognition Access Control System

Appearance-Invariant Face Authentication System

Robustness

.... **system continues to recognize faces accurately even when conditions are not ideal.**

A robust face recognition system should work reliably despite:

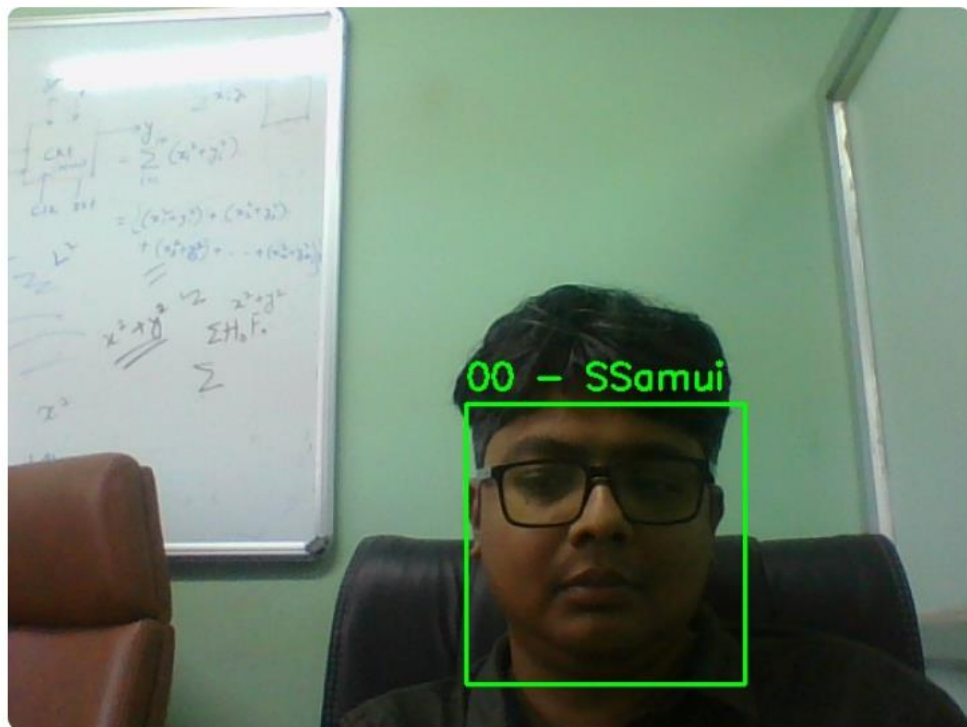
- 1.Lighting changes** — works in bright sunlight, dim rooms, or shadows.
- 2.Pose variations** — recognizes the same person from different angles (front, left, right).
- 3.Facial expressions** — identifies correctly whether the person is smiling, frowning, or neutral.
- 4. Aging or appearance changes** — still matches older vs. newer images.
- 5.Camera quality differences** — performs well across various resolutions and devices.
- 6.Background clutter or crowd** — isolates and matches the right face in complex scenes.

To solve this problem.....

- Do we need to build a discriminative classifier?
- Suppose a Face recognition system (Classifier) for face classification was developed. Is it enough to solve the problem?

Core Tasks:

- 1. Face Detection:** Locate faces in the camera frame.
- 2. Face Encoding:** Convert each face into a numerical vector (128-dimensional embedding).
- 3. Face Recognition/Verification:** Compare the new face embedding with stored ones to identify or verify the person.
- 4. Enrollment:** Register a new person in the system using a single image (one-shot learning).



Single Face



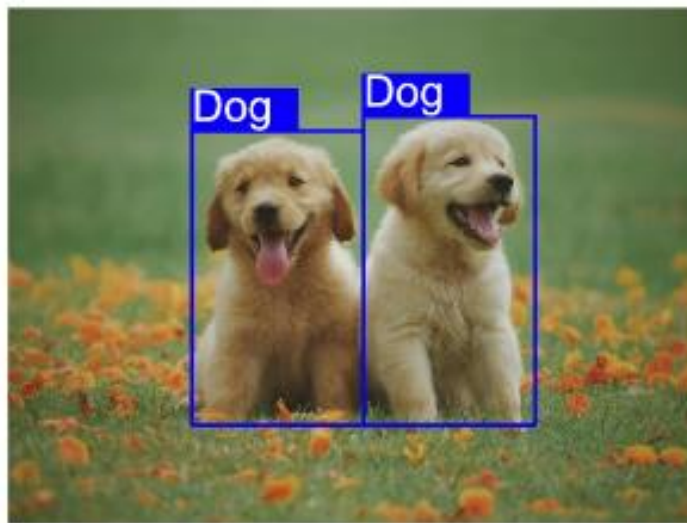
Multiple Faces

The three major sub-problems of computer vision.

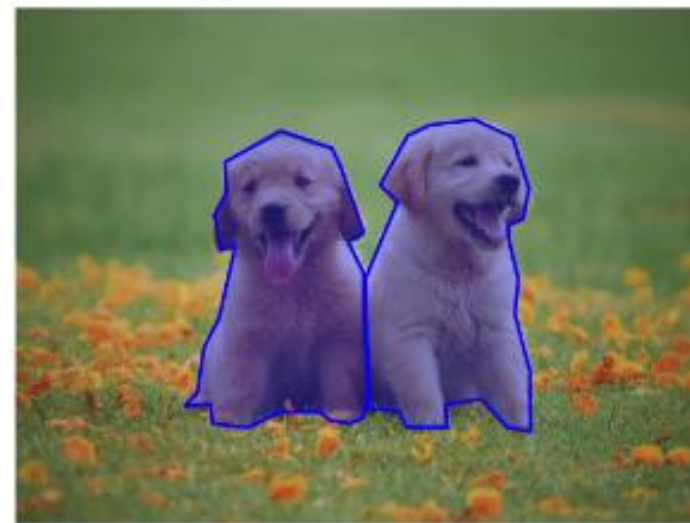
Classification



Detection



Segmentation



What is Object Detection?

Object detection is a computer vision technique for locating and classifying multiple objects within an image or video. Unlike image classification (which assigns one label to the whole image), object detection:

- Draws **bounding boxes** around each detected object
- Assigns a **class label** to each box (e.g., “car”, “person”)
- Returns a **confidence score** indicating detection certainty

Image
Classification

V / S

Object
Detection

- Object Classification

- Object Classification
- Object Localization

Object Detection



Target Class:
Emergency Vehicle

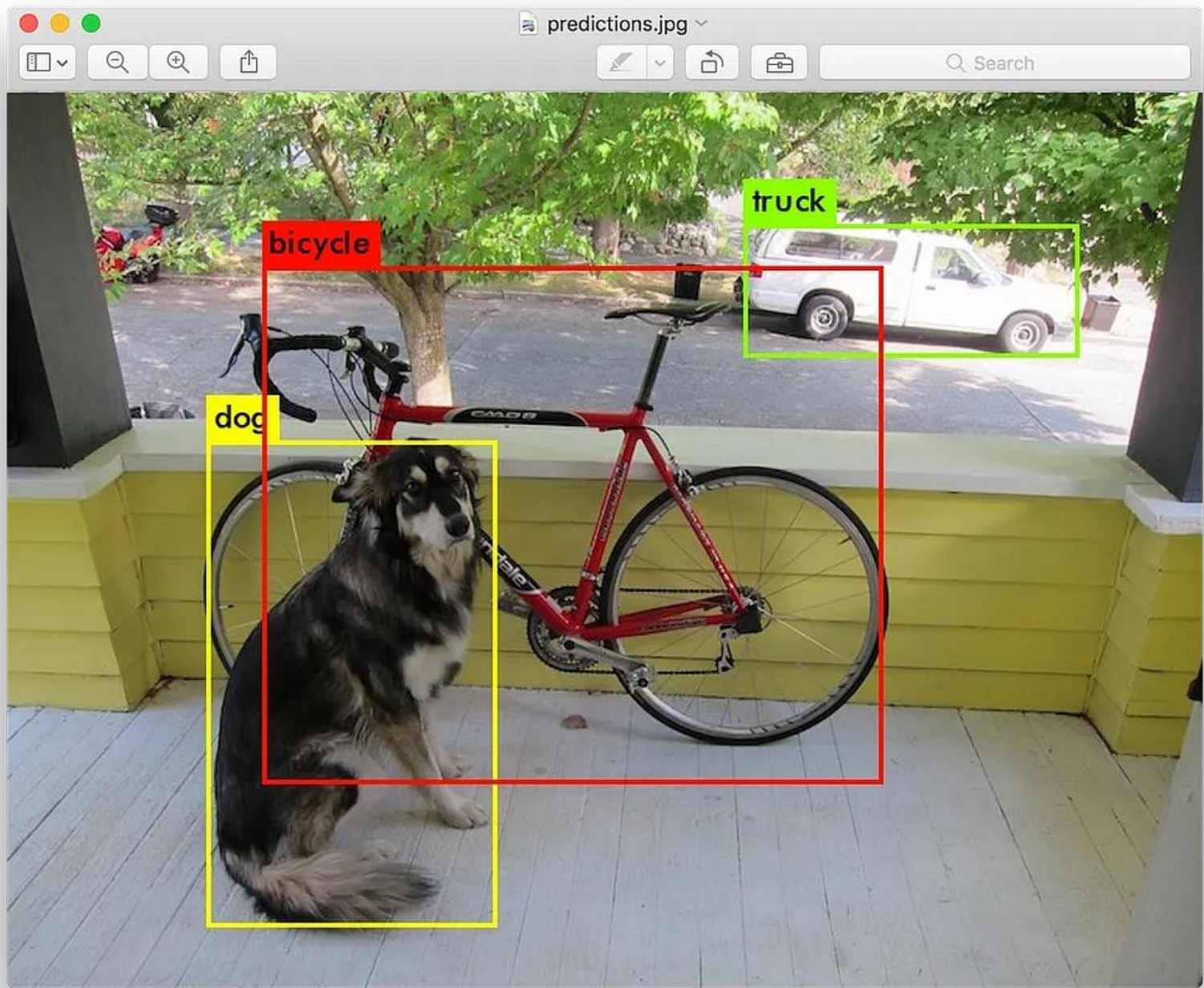
Predict Target Class

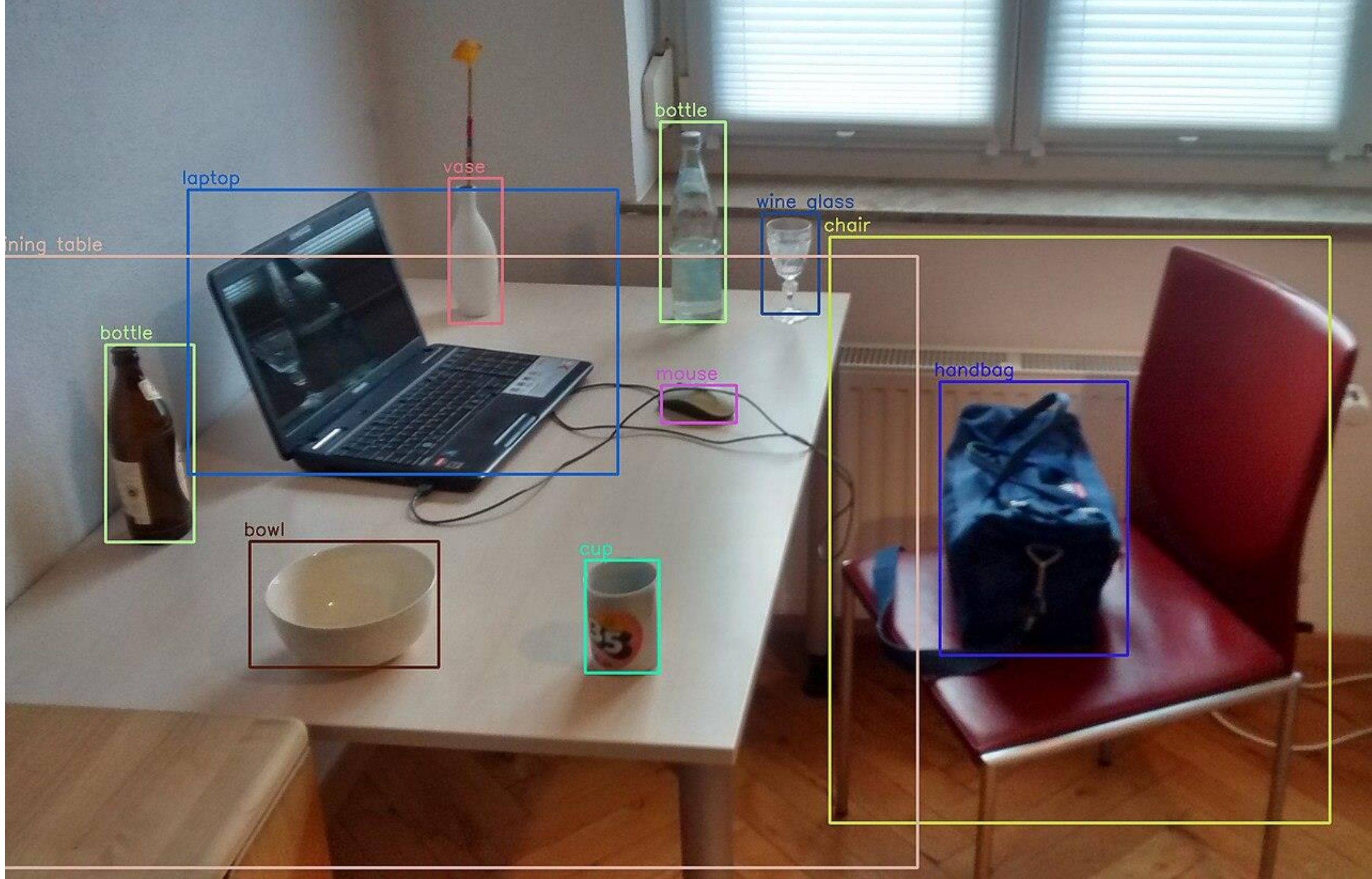
Predict Object Location

Is an Object Present
in the image?

Where is the Object
located?

What is the Object?





laptop

vase

bottle

wine glass

chair

mouse

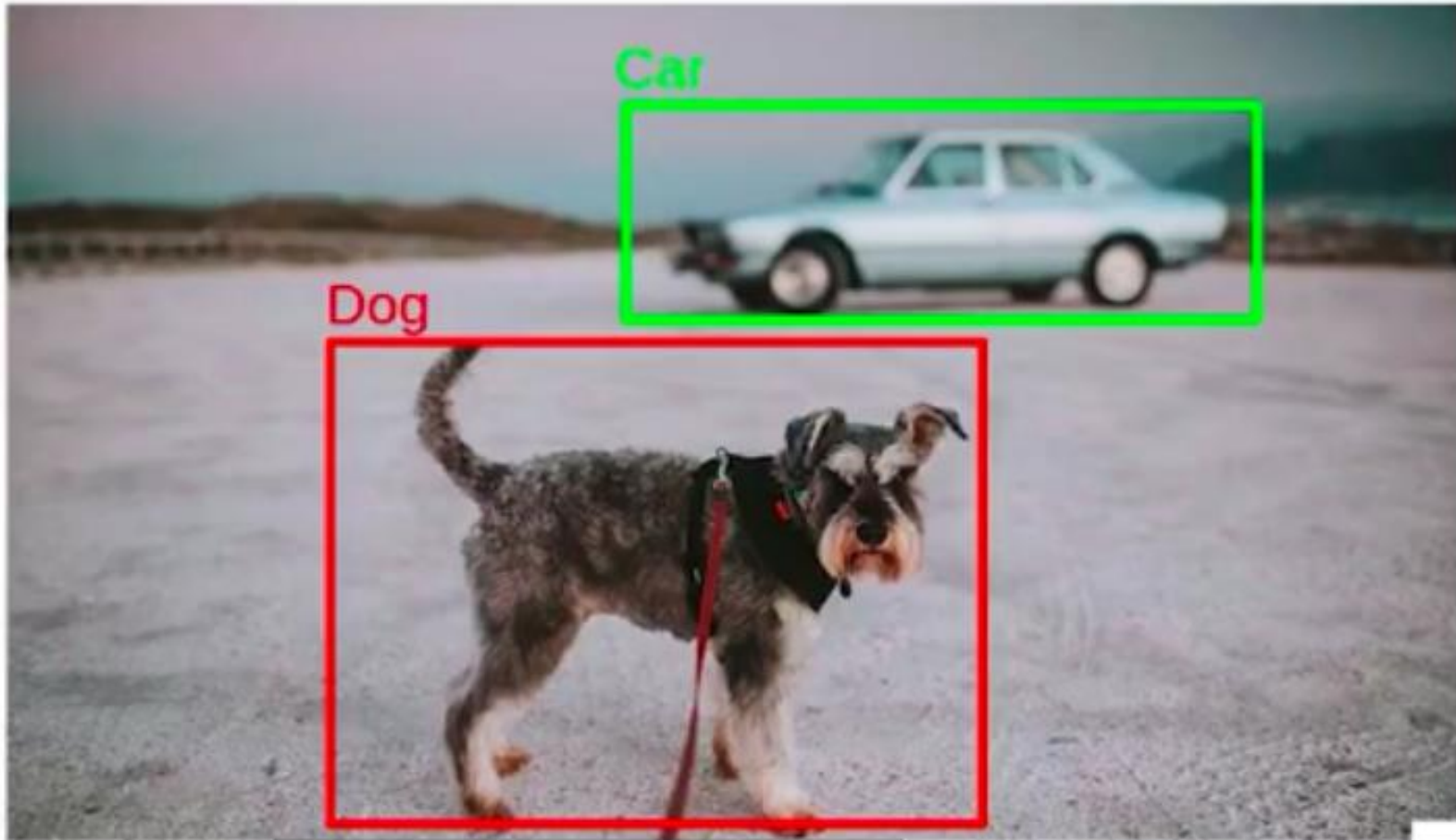
handbag

bottle

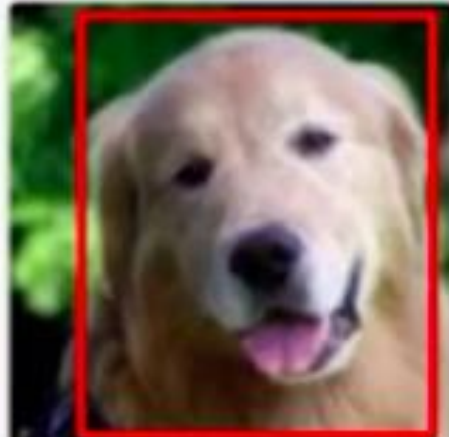
bowl

cup

dining table







Object detection typically involves **two key components**:

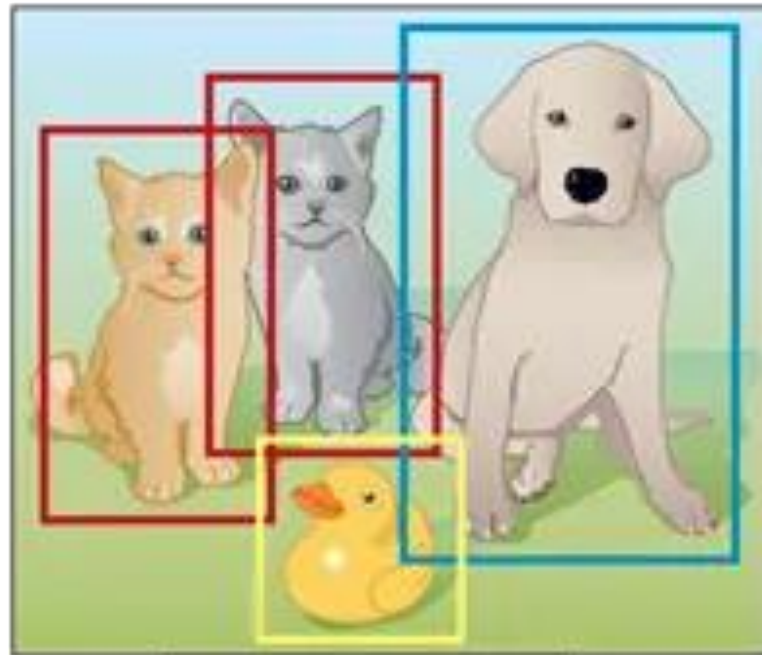
1. Identifying **where objects are** in an image — localization.
2. Determining **what those objects are** — classification.

Image classification

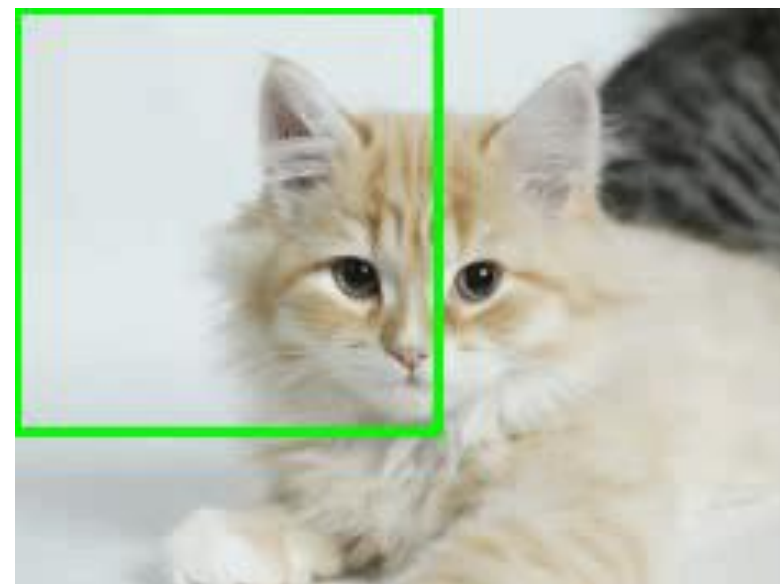
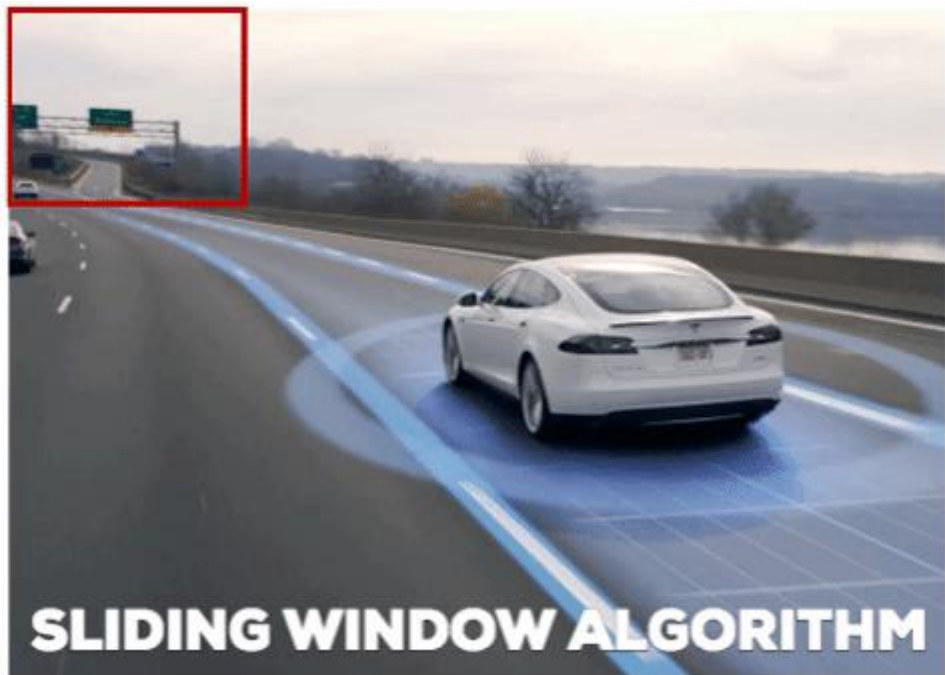


Cat

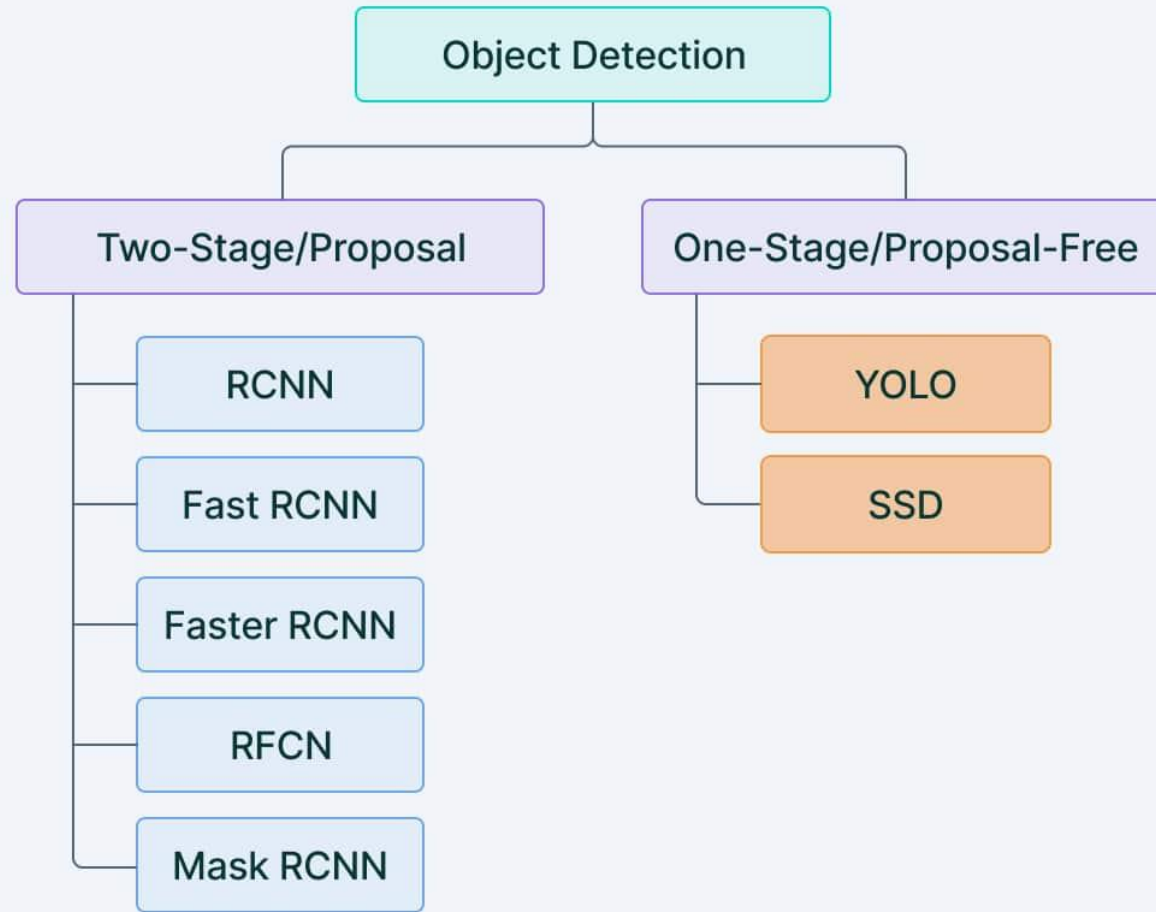
Object detection
(classification and localization)



Cat, Cat, Duck, Dog



One and two stage detectors



YOLO (You Only Look Once)

YOLO: Release Dates Timeline



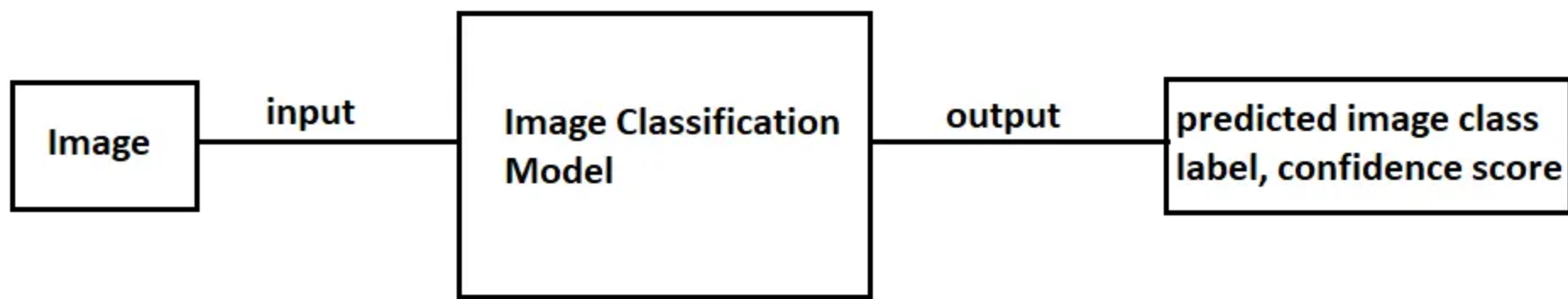
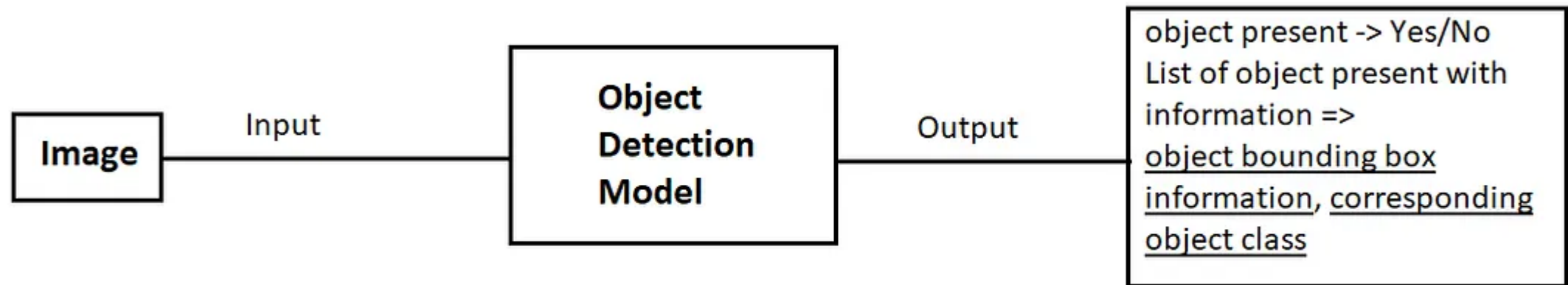


Image Classification Abstract Idea



Object Detection Model Abstract Idea

Image Classification

Is this a dog or a person?



Neural
Network
Output

Dog = 1
Person = 0

Object Localization

Where exactly is the dog in
this image?



Neural
Network
Output

Dog = 1
Person = 0

+

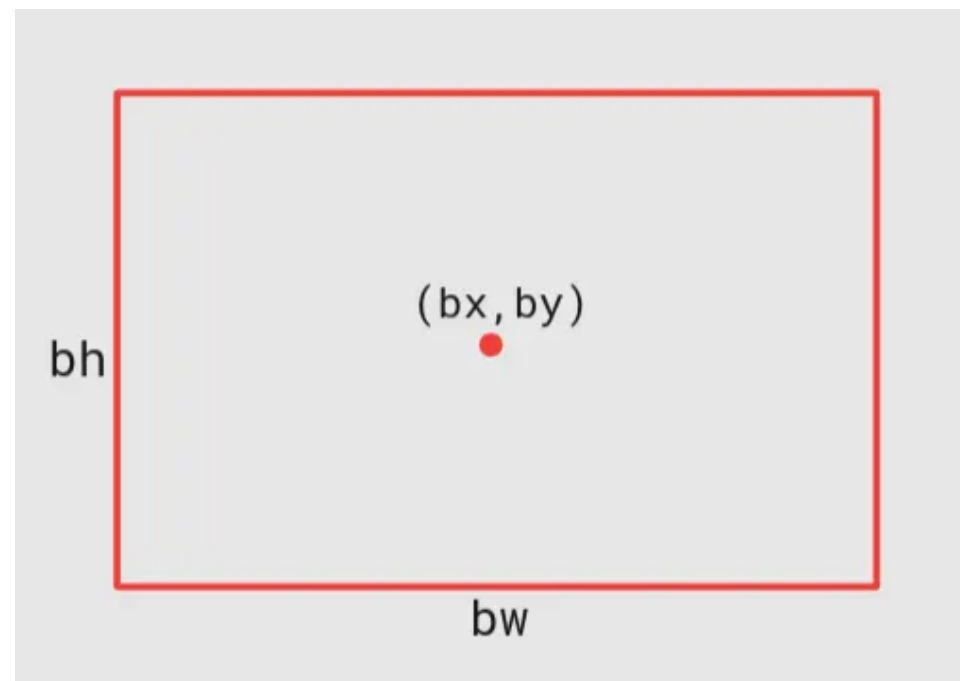
Bounding
Box

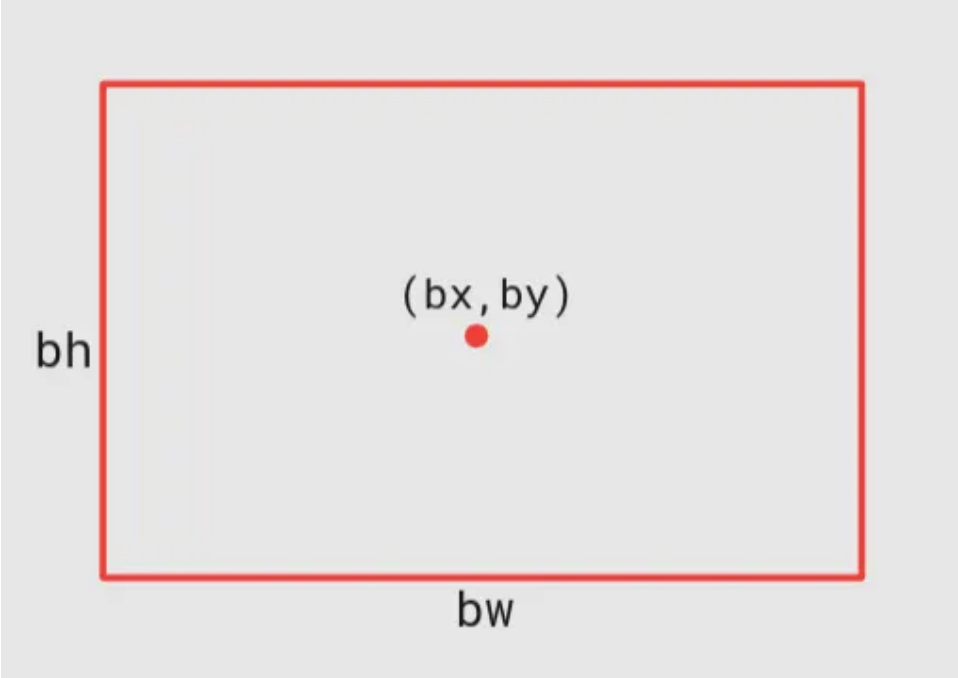
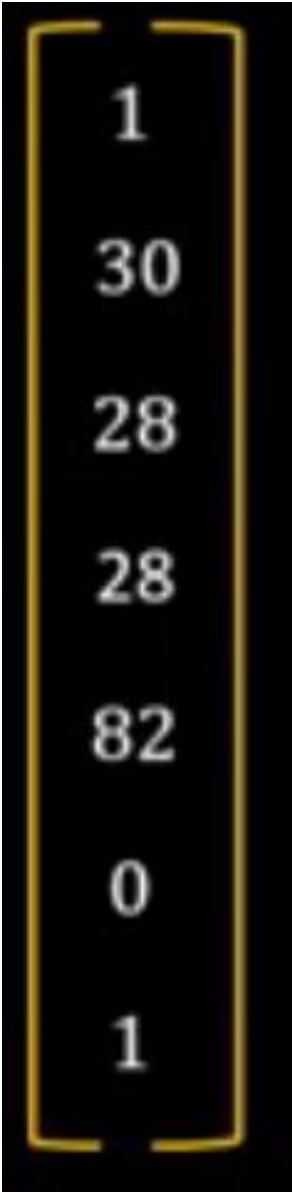
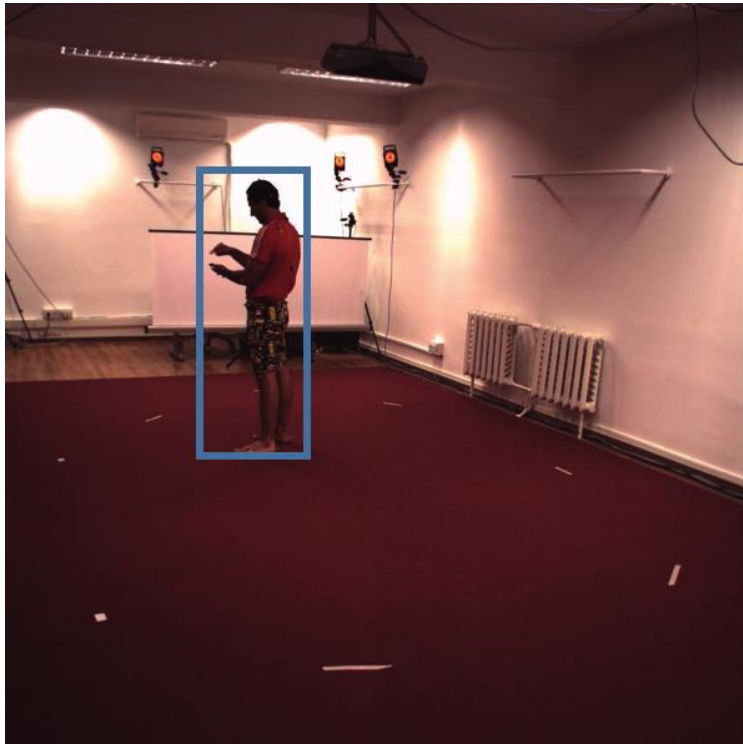
Object Localization



P_c	1
B_x	50
B_y	70
B_w	60
B_h	70
C_1	1
C_2	0

$C_1 = \text{Dog class}$
 $C_2 = \text{Person Class}$





No relevant object....



X_train



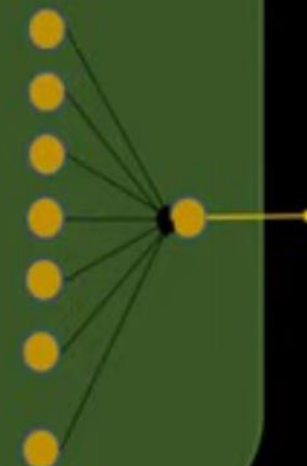
y_train

p_c	1
B_x	50
B_y	70
B_w	60
B_h	70
C_1	1
C_2	0

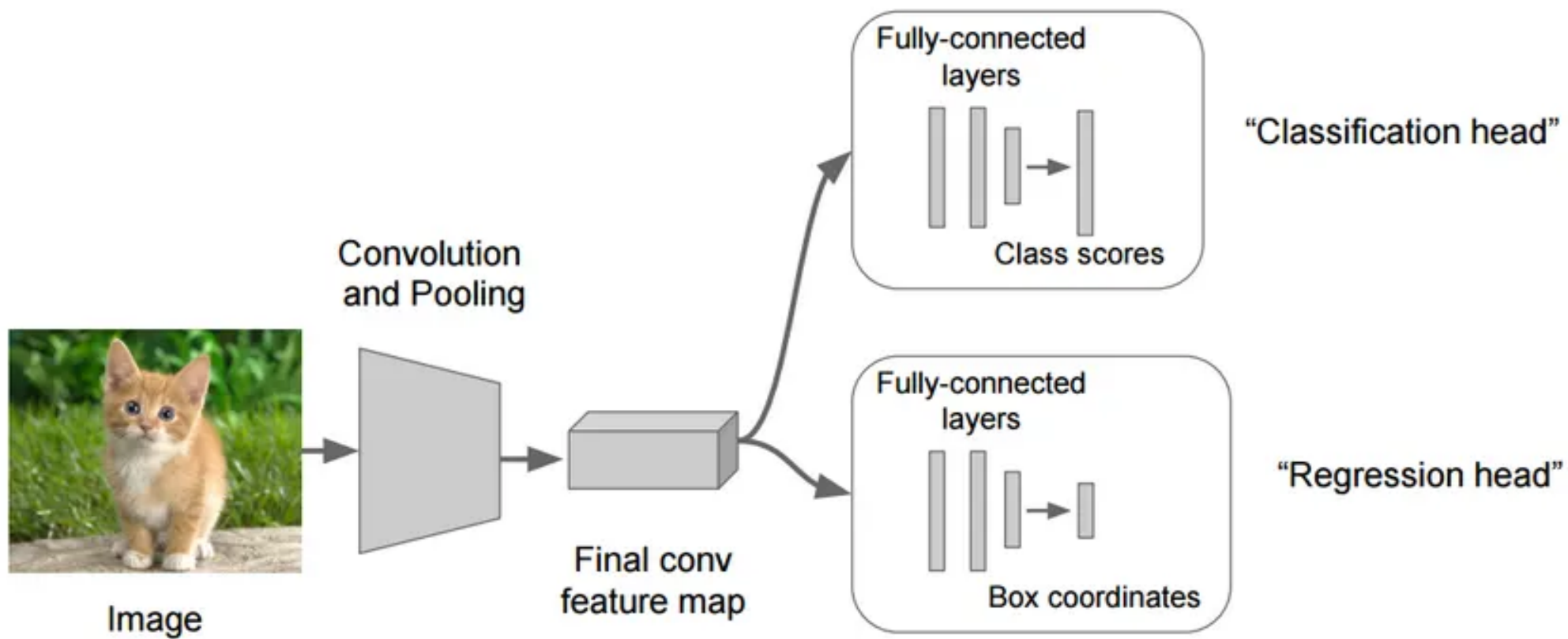
1
30
55
28
82
0
1

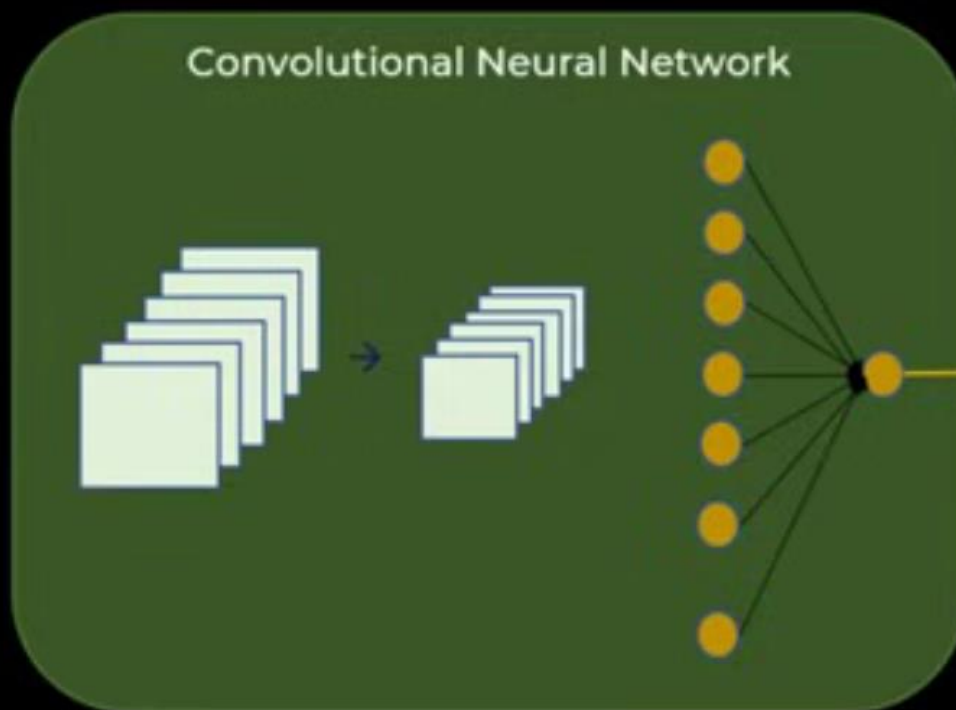
0
-
-
-
-
-
-

Convolutional Neural Network



p_c
B_x
B_y
B_w
B_h
C_1
C_2





1
25
57
30
42
1
0



Step-by-Step Workflow

- Step 1: Capture or Load an Image/Frame
- Step 2: Detect Faces
- Step 3: Encode the Face
- Step 4: Store (Enroll) a New Person – One-Shot Learning
- Step 5: Recognition or Verification during Access

One-Shot Learning for Face Recognition



- **One-shot learning** means recognizing a person after seeing just **one image** of them during enrollment.
- Unlike traditional face recognition systems (which need dozens or hundreds of images per person), a one-shot system learns to compare new faces with stored ones rather than retraining a classifier.

This is usually done through **metric learning**:

- The network outputs an **embedding vector** for each face (e.g., 128-D).
- During training, embeddings of the *same person* are made **close** in space, while embeddings of *different people* are made **far apart**.

Function Type	Formula	Range / Meaning
Euclidean distance	$d(x_i, x_j) = \ f(x_i) - f(x_j)\ _2$	Smaller = more similar
Cosine similarity	$S(x_i, x_j) = \frac{f(x_i) \cdot f(x_j)}{\ f(x_i)\ \ f(x_j)\ }$	Larger = more similar (ranges -1 to 1)

Why It Is Called *Metric Learning*

Because the model learns a **metric space** — a space where:

- Distance corresponds to **semantic similarity** (same identity = small distance),
- Different identities are **far apart**.

Formally, the training enforces the **properties of a metric**:

1. $d(a, b) \geq 0$ (non-negative)
2. $d(a, b) = 0 \Leftrightarrow a = b$

How It Works Step-by-Step

1. Pre-train a face embedding model

Example: FaceNet, Dlib, or `face_recognition` library (based on ResNet trained on millions of faces).

2. Enrollment (One-Shot Step)

- Take one photo per new person.
- Compute and store its embedding vector.

3. Recognition / Verification

- Capture a new live face.
- Compute its embedding.
- Compare with stored embeddings using distance metric (L2 or cosine).
- If distance < threshold → same person; else → unknown.